

Commento al testo e alle soluzioni proposte

Esercizio di programmazione concorrente

Questo esercizio non presentava particolari difficoltà, anche perché il testo stesso diceva cosa fare.

I punti essenziali della soluzione sono:

1. il main fa sleep subito dopo aver lanciato i thread, quindi quando arriverà l'interrupt da un thread verrà svegliato con una eccezione: a quel punto deve mandare un interrupt a ciascun thread.
2. I thread devono mandare un interrupt al main quando trovano la soluzione, e devono reagire alla ricezione di un interrupt terminando.

L'unico dettaglio degno di nota è che i thread, che sono per la maggior parte del tempo impegnati a fare i calcoli, non possono reagire immediatamente all'interrupt. Nella soluzione proposta i thread controllano la presenza di interrupt tra due sessioni di calcolo successive. Volendo realizzare un sistema più reattivo, si può inserire in controllo dell'interruzione anche dentro al metodo di calcolo.

Esercizi di programmazione distribuita

In questo caso (sia che si usino i socket direttamente, sia che si usi RMI) bisogna considerare che non c'è la possibilità di inviare interrupt, ma bisogna usare i meccanismi di comunicazione disponibili.

Realizzare il sistema come specificato per l'esercizio di programmazione concorrente sarebbe complesso. Ad es., nel caso di uso dei socket, lato client bisognerebbe creare un thread dedicato ad ascoltare l'eventuale arrivo di messaggi di "interrupt" dal server.

Per semplificare il sistema, il testo dell'esercizio di programmazione con socket suggeriva di alternare (lato client) la fase di calcolo e la fase di comunicazione in cui il client comunica al server se ha trovato la soluzione o meno, e il server comunica al client se proseguire nei calcoli o terminare.

Lato server, essendo il server stesso continuamente in attesa di nuove connessioni, occorre trovare il modo di interrompere di tanto in tanto quest'attesa per verificare se la soluzione è stata trovata. Il modo più semplice di fare ciò consiste nel settare un timeout sull'attesa di connessioni (come già visto a esercitazione nel caso dell'asta on-line).

Nel caso di RMI, la situazione è molto più semplice: basta realizzare sia client sia server come oggetti remoti. Il client avrà il main continuamente impegnato a cercare la soluzione, ma offrirà al server un metodo per segnalare che si può terminare.

Il server offre al client il metodo che serve a segnalare la soluzione, mentre il main verifica periodicamente se la soluzione è stata trovata: quando la verifica ha successo, chiama i metodi di terminazione dei client.